

Day 29: Create Product Dictionary

Day 29 — Create Product Dictionary

1. Day Number

Day 29

2. Topic Name

Product Dictionary

3. Connection to Previous Learning

Earlier, you learned Python dictionaries using examples such as **user profiles**.

For example, a profile might contain:

```
profile = {  
    "name": "Radhe",  
    "city": "Bangalore"  
}
```

Today, you will use exactly the same dictionary concept for **product data**.

Instead of storing:

```
name  
city
```

you will store:

```
product name  
price  
stock
```

This is a small step toward understanding how real applications represent products in inventory systems, shopping applications, and APIs.

4. Important Topics

Focus on these concepts today:

- Dictionary
- Dictionary keys
- Dictionary values
- name
- price
- stock
- Accessing dictionary values
- Passing a dictionary to a function
- Printing product information

5. Foundational Notes

What is a dictionary?

A dictionary stores information in **key-value pairs**.

Think about a product:

```
Laptop  
  
Name → Laptop  
Price → 50000  
Stock → 10
```

Python can represent this naturally using a dictionary.

Conceptually:

```
product  
├── name → Laptop  
├── price → 50000  
└── stock → 10
```

Each piece of information has two parts:

```
key → value
```

For example:

```
"name" → "Laptop"  
"price" → 50000  
"stock" → 10
```

Dictionary structure

The general syntax is:

```
variable_name = {  
    "key1": value1,  
    "key2": value2  
}
```

A dictionary uses:

- { } to contain the data
- : between a key and value
- , between different key-value pairs

Why use a dictionary for a product?

Without a dictionary, you could create several independent variables:

```
name = "Laptop"
price = 50000
stock = 10
```

This works, but those variables are separate.

A dictionary groups them together:

```
Product
├── name
├── price
└── stock
```

That makes it clear that all three values belong to the **same product**.

Accessing dictionary values

Suppose a dictionary contains:

```
"name" → "Keyboard"
"price" → 1500
"stock" → 8
```

You can access a value using its key.

General syntax:

```
dictionary_name["key"]
```

For example:

```
some_dictionary["name"]
```

means:

| Give me the value stored under the key "name".

6. Easy Example

Imagine we want to represent a book.

```
book = {
    "title": "Python Basics",
    "pages": 200
}
```

We can access individual values:

```
print(book["title"])
print(book["pages"])
```

Output:

The same idea applies to your product:

```
product["name"]  
product["price"]  
product["stock"]
```

7. Problem Statement

Create one product dictionary containing:

- name
- price
- stock

Then print the product details.

For example, conceptually your product could represent:

```
Product Name : Mouse  
Price       : 800  
Stock       : 15
```

Do not worry about multiple products yet.

You only need to handle **one product**.

8. Concepts Used

You will practice:

- Variables
- Strings
- Integers or floats
- Dictionaries
- Key-value pairs
- Dictionary access
- Functions
- Function parameters
- print()

The main new connection is:

```
Dictionary  
  ↓  
Represent an object  
  ↓  
Product
```

```
↓  
name + price + stock
```

9. Thought Process

Before writing Python, think about the problem in plain English.

Step 1: What information describes the product?

You need:

```
name  
price  
stock
```

Step 2: Do these values belong together?

Yes.

They all describe one product.

Therefore, a dictionary is a good choice.

Step 3: What should the dictionary keys be?

Think about keys such as:

```
"name"  
"price"  
"stock"
```

Step 4: What values should those keys contain?

For example:

```
"name" → some product name  
"price" → some numeric price  
"stock" → some quantity
```

Step 5: How should the details be displayed?

You need to access each value using its key and print it.

Step 6: Since we want a functional approach

Instead of putting all printing logic directly in the main program, create a function whose responsibility is:

```
receive product  
  ↓  
read product values  
  ↓  
print product details
```

This gives you early practice with separating **data** from **behavior**.

10. Pseudocode

```
START

Create a dictionary called product

Store:
    name
    price
    stock

Create a function to display product details

Function receives product dictionary

Inside function:
    print product name
    print product price
    print product stock

Call the function and pass product

END
```

Flow:

```
Create Product Dictionary
    |
    ▼
{
  name,
  price,
  stock
}
    |
    ▼
Pass dictionary to function
    |
    ▼
Read values using keys
    |
    ▼
Print product details
```

11. Suggested Solving Approach — Functional Approach

Use two small responsibilities.

Part 1 — Create the data

Create the product dictionary.

Conceptually:

```
product = {  
    name information,  
    price information,  
    stock information  
}
```

Part 2 — Create a function

Create something like:

```
def display_product(product):  
    # access dictionary values  
    # print them
```

Then call the function:

```
product  
  ↓  
display_product(...)  
  ↓  
product details
```

The important idea is:

```
DATA  
product dictionary  
  
  ↓  
  
FUNCTION  
display_product()  
  
  ↓  
  
OUTPUT  
product information
```

This is better practice than mixing everything together as your programs grow.

12. Easy Edge Cases

For today's exercise, consider only two simple edge cases.

Edge Case 1 — Stock is 0

Example:

```
name → "Keyboard"  
price → 1500  
stock → 0
```

0 stock is valid.

It simply means:

```
The product currently has no available units.
```

Your program should still print:

```
Stock: 0
```

Do not treat 0 as missing data.

Edge Case 2 — Price is 0

Example:

```
name  → "Sample"
price → 0
stock → 5
```

For this exercise, 0 is also a valid value.

Your program should still print it correctly.

Important beginner lesson:

```
0
```

is different from:

```
None
```

and different from a missing dictionary key.

For Day 29, you do not need to validate these cases. Just make sure your code can display them.

13. Expected Input

You do **not** need `input()` from the user today.

Create the dictionary directly in Python.

Example conceptual input:

```
name  = "Mouse"
price = 800
stock = 15
```

These values should be stored together inside one dictionary.

14. Expected Output

Your output should look similar to:

```
Product Name: Mouse
Price: 800
Stock: 15
```

Exact formatting is not important.

The important requirement is that all three values are displayed.

For the stock edge case:

```
Product Name: Mouse
Price: 800
Stock: 0
```

For the price edge case:

```
Product Name: Sample Product
Price: 0
Stock: 5
```

15. Hint Only

Start with the dictionary structure:

```
product = {
    "name": ...,
    "price": ...,
    "stock": ...
}
```

Then create a function:

```
def display_product(product):
    ...
```

Inside the function, remember that dictionary values can be accessed using:

```
product["some_key"]
```

So ask yourself:

```
How do I get the name?
How do I get the price?
How do I get the stock?
```

Then use those values with `print()`.

Your target structure

```
product dictionary
  ↓
display_product(product)
  ↓
access name
access price
access stock
  ↓
print details
```

That is enough information to solve Day 29 without seeing the complete solution.